

IntegrityFlow

Module Explanation Guide

Panel preparation for Rao Umair Ahmed

A companion to the revised 10-module form. Prepared from the current source code on 7 October 2026. This guide explains the form; it does not change the implementation.

How to answer a panel question

Use four steps: explain the purpose, trace the workflow, point to the code, and show a small demonstration. Use the exact claim boundaries on each module page when an evaluator asks about guarantees.

Overall architecture

Candidate renderer <- Electron IPC through preload.js -> Electron main process <- localhost HTTP on dynamic ports -> Python monitors. Events and evidence are persisted locally and sent to the central Node/Express server. MongoDB stores session/event records; storage providers hold files. Socket.IO and REST connect the server to the examiner dashboard. Answer submission is a separate renderer-to-server request.

Your assigned responsibility

Rao Umair Ahmed: desktop app and environment control, evidence capture, answer submission, and the shared Electron-Python bridge. Understand how AI and backend/dashboard modules connect to your work; detailed model/scoring ownership can be shared with the assigned teammate.

Important distinctions

- An exam is the teacher-created assessment; a session is one candidate's participation. An event belongs to that session.
- Desktop screenshots show software activity; webcam frames show the camera scene. An audio clip is separate event evidence.
- A detection is a review signal. Severity is an event weight; risk is a candidate review score, not a cheating probability.
- Physical-lab mode is not development mode. Self-check is not continuous enforcement.
- Removing the standalone voice module from the form did not remove its implementation or the remaining voice references in self-check.

Code references

Paths are relative to D:\BS_FYP Project\IntegrityFlow. Each file:line points to a relevant function or route in the current working tree. Line numbers can move after edits. No secrets or credentials are included.

1. Secure Environment Enforcement

Say to the panel: I built the candidate-side environment controls: permitted tools remain usable, while prohibited applications and old documents generate evidence-backed enforcement events.

Windows candidate desktop application using Electron.

Purpose: Provide one desktop entry point for the exam and native Windows integration.

How it works: Electron main creates the window, starts Python and coordinates login, consent, self-check and exam mode. contextIsolation is enabled and nodeIntegration is disabled; preload exposes selected IPC methods.

Where to show it: candidate-app/electron/main.js:70 - async function createWindow
candidate-app/electron/preload.js:10 - contextBridge.exposeInMainWorld

Application whitelist enforcement with examiner-permitted tools and process monitoring.

Purpose: Allow required exam software while restricting unrelated tools.

How it works: The teacher stores executable names with the exam. Python expands known app aliases and inspects candidate-owned processes. Protected OS/app processes are excluded. Allowed tools are inspected for old files; prohibited processes enter the enforcement handler.

Where to show it: candidate-app/ai-module/whitelist_enforcer.py:214 - def _parse_allowed_apps
candidate-app/ai-module/whitelist_enforcer.py:359 - def _monitor_loop

Clipboard and display restrictions, USB-storage monitoring and pre-existing-file checks.

Purpose: Reduce copying and use of extra screens, external storage or prepared notes.

How it works: The renderer blocks clipboard actions and context menus; Electron clears the clipboard and monitors displays. Python detects USB disks, captures evidence and requests safe removal. An old document is identified using modification time and file/title/Recent observations; detection can close the allowed application.

Where to show it: candidate-app/renderer/examScreen.js:824 - ['copy', 'cut', 'paste', 'contextmenu']
candidate-app/ai-module/whitelist_enforcer.py:679 - def _inspect_allowed_app
candidate-app/ai-module/usb_monitor.py:193 - def _eject_storage

Explicit permitted-file exceptions and online/physical-lab exam modes.

Purpose: Permit approved material and adapt monitoring to the exam setting.

How it works: The old-file guard exempts normalized approved paths. Online mode starts camera and voice monitoring; physical-lab mode bypasses them while environment monitoring remains. APP_MODE=dev is a separate development setting.

Where to show it: candidate-app/ai-module/whitelist_enforcer.py:784 - def _is_permitted_file
candidate-app/ai-module/main.py:116 - if is_physical_lab:

Demonstration / example

Permit Notepad; create a new file, then open an old unapproved file. Show the alert and application closure. Use a spare USB to demonstrate safe removal.

Claim boundary - explain accurately

Do not call this an unbreakable kiosk: the current window is not configured as a complete OS lockdown. File timestamps are a practical policy, not proof of content age. USB removal may be vetoed; app closure can discard other open documents.

2. AI Monitoring

Say to the panel: The camera is processed locally. We report sustained visual events with evidence, rather than treating one uncertain frame as proof of cheating.

Local webcam processing using OpenCV and MediaPipe face landmarks when available.

Purpose: Measure camera events without uploading a continuous video stream.

How it works: OpenCV reads webcam frames. MediaPipe Tasks with a local face-landmarker model is preferred, with legacy FaceMesh and OpenCV Haar fallbacks. Landmark coordinates feed OpenCV pose estimation; a fallback has different accuracy.

Where to show it: candidate-app/ai-module/ai_monitor.py:40 - class AIMonitor
candidate-app/ai-module/ai_monitor.py:552 - def _check_head_pose

Head-turn, missing-face, multiple-face/person and camera-obstruction signals.

Purpose: Flag sustained absence or an unusual camera scene.

How it works: Head pose uses six landmarks, solvePnP, geometry checks, a forward baseline and direction timers. Face counts have persistence checks. A separate lighting/occlusion detector evaluates darkness and image variation; additional person checks also use object inference.

Where to show it: candidate-app/ai-module/ai_monitor.py:662 - def _observe_head_pose
candidate-app/ai-module/ai_monitor.py:755 - def _record_face_count
candidate-app/ai-module/services/lighting_occlusion_detector.py:10 - class CameraOcclusionDetector

YOLO26n ONNX inference for configured phone/book checks.

Purpose: Detect selected prohibited visible objects.

How it works: ONNX Runtime executes the local model on CPU. Frames are resized/padded for inference, predictions are decoded and filtered by class/confidence. Spatial and repeated-frame checks suppress isolated detections; a candidate phone is checked again promptly.

Where to show it: candidate-app/ai-module/ai_monitor.py:767 - def _check_objects

Calibration and confidence/persistence filters; alerts support examiner review.

Purpose: Reduce transient false positives and make alerts reviewable.

How it works: A stable forward pose establishes a neutral baseline. Angle limits, confidence checks and sustained conditions precede an event. Each accepted signal gets a session, timestamp and supporting frame. The examiner may confirm or dismiss it.

Where to show it: candidate-app/ai-module/ai_monitor.py:716 - def _track_head_direction
dashboard/src/components/AlertFeed.jsx:111 - handleQuickReview

Demonstration / example

Wait for the neutral-position calibration log, turn left/right, return forward, and show the resulting webcam evidence. Hold a phone in view and compare with an empty frame.

Claim boundary - explain accurately

Head direction is not precise eye-gaze tracking. The project runs pretrained models; do not claim custom model training or measured accuracy without your own test results. Headphones are not an implemented detection class.

3. Evidence Capture & Delivery

Say to the panel: Every alert should explain what happened, when it happened and which candidate session it belongs to. Evidence and delivery are separate from scoring.

Event-triggered desktop screenshots for system events and webcam frames for visual events.

Purpose: Capture the right view for the event.

How it works: MSS captures the desktop for app/file/USB events. WebcamCaptureProvider saves the supplied camera frame for visual events. Desktop evidence cannot show the face scene; webcam evidence cannot establish which document was open.

Where to show it: candidate-app/ai-module/services/capture/__init__.py:24 - def capture_screenshot
candidate-app/ai-module/services/capture/__init__.py:30 - def capture_webcam_frame

Timestamped JPEG evidence linked to the candidate session and violation.

Purpose: Keep evidence attributable and distinguish separate incidents.

How it works: Capture providers create timestamped JPEG names with session/type information. The event carries sessionId, eventId, timestamp, details and evidence paths. The server stores the file and records the stored URL on the violation.

Where to show it: candidate-app/ai-module/services/capture/MssCaptureProvider.py:37 - def capture
server/src/routes/violations.js:108 - Process screenshot file

Local durable queues, stable event IDs and retries through Python and Electron.

Purpose: Preserve events through network failures and restarts.

How it works: Python persists events before handoff; Electron persists queued server uploads using SQLite or JSON fallback. Original timestamps and event IDs survive retry. The server recognizes duplicate delivery rather than creating another copy of the same event.

Where to show it: candidate-app/ai-module/services/violation_delivery.py:29 - def enqueue
candidate-app/electron/ipc/violationBuffer.js:121 - function enqueue
candidate-app/electron/ipc/pythonBridge.js:270 - startBufferRetryLoop

Server evidence storage and retrieval, including valid delayed evidence after exam completion.

Purpose: Allow review even when upload arrives late.

How it works: The storage service selects local or Cloudinary storage. Capture-time/session-window checks distinguish valid delayed events from invalid ones. receivedLate identifies late arrivals; late evidence needs explicit review before affecting risk. Protected evidence URLs are checked against teacher ownership/admin access.

Where to show it: server/src/routes/violations.js:103 - receivedLate:
server/src/services/storage/index.js:21 - module.exports
server/src/index.js:36 - app.use('/uploads')

Demonstration / example

Generate an alert, briefly disconnect the server/network, show the queued count, restore connectivity and show delivery with the original timestamp. Explain the screenshot versus webcam frame.

Claim boundary - explain accurately

Queues improve resilience but are not an absolute guarantee against disk failure or missing evidence. Continuous video is not stored. Delayed alerts should not be presented as fresh live incidents.

4. Candidate Self-Check System

Say to the panel: Self-check catches setup problems before an exam starts. It is a readiness workflow, not the same as continuous exam enforcement.

Step-by-step camera/face calibration, microphone and reference-voice checks.

Purpose: Verify that the candidate can be monitored with usable inputs.

How it works: The self-check screen opens the camera, checks centered face alignment and microphone input, and records a voice reference plus confirmation. This capture helps readiness; continuous head-pose calibration occurs when the exam camera monitor starts.

Where to show it: candidate-app/renderer/selfCheck.js:153 - function startFaceAlignmentTracking
candidate-app/ai-module/voice_monitor.py:145 - def set_reference_voice

Application, USB-storage and display verification before exam entry.

Purpose: Find blocked software, external storage and extra monitors early.

How it works: The screen invokes preload methods checkApps, checkUsbDrives and getDisplayCount. The Begin button depends on all required checks passing. Self-check lists USB devices; ejection/enforcement occurs during an active exam.

Where to show it: candidate-app/renderer/selfCheck.js:84 - function updateBeginButton
candidate-app/electron/preload.js:24 - checkUsbDrives:

Physical-lab mode bypasses camera and voice checks.

Purpose: Avoid treating a supervised shared lab as a private online camera/audio environment.

How it works: The selected exam type controls which checks are required. Python also bypasses the corresponding camera and voice monitors in physical_lab mode, so the bypass is not just a cosmetic UI change.

Where to show it: candidate-app/renderer/selfCheck.js:34 - physical_lab
candidate-app/ai-module/main.py:116 - if is_physical_lab:

Demonstration / example

Open self-check with an extra display or USB attached and show the failed readiness check. Remove it, rerun the check and explain how Begin becomes available.

Claim boundary - explain accurately

Removing the standalone voice module from the form did not remove voice-related self-check code or the voice feature. The form still mentions microphone/reference voice here; explain this honestly if asked.

5. Violation Severity Scoring Engine

Say to the panel: The server converts eligible violation records into a review-priority score. It helps the examiner choose whom to inspect first; it is not a verdict.

Server-side severity weighting and time-decayed aggregation of violation records.

Purpose: Combine severity and recency into a consistent session score.

How it works: The engine fetches eligible violations and calls DecayScoringStrategy. Recent weight decays exponentially with an approximately 30-minute half-life. A persistent baseline means past eligible violations do not simply vanish. The result is normalized and capped at 100.

Where to show it: server/src/scoring/severityEngine.js:4 - calculateRiskScore
server/src/services/scoring/DecayScoringStrategy.js:24 - calculateScore

Per-candidate risk score and examiner review priority.

Purpose: Direct attention to candidates with more concerning records.

How it works: Scores are associated with the candidate session. The risk-score endpoint and socket updates supply dashboard badges; Priority Queue lists unreviewed events across active exams. Event severity and candidate risk are distinct values.

Where to show it: server/src/routes/riskScore.js:8 - router.get('/:sessionId'
dashboard/src/components/RiskScoreBadge.jsx:4 - RiskScoreBadge
dashboard/src/components/PriorityQueue.jsx:192 - export default function PriorityQueue

Review decisions affect scoring; risk is not a probability of cheating.

Purpose: Let human decisions remove unsupported signals.

How it works: Dismissed records are excluded by the engine; eligible confirmed/pending events follow its scoring rules. Late evidence needs explicit confirmation to score. Review actions trigger updated records and score broadcasts. A score of 80 is not an 80% cheating probability.

Where to show it: server/src/scoring/severityEngine.js:5 - dismissed
server/src/routes/violations.js:382 - /review

Demonstration / example

Show a candidate score, dismiss a relevant event and show the changed score. Use an isolated demo session so other violations do not obscure the change.

Claim boundary - explain accurately

Visual grouping combines repeated alerts for easier reading; it does not automatically group them in the scoring engine. Do not say incident-level score deduplication is implemented.

6. Examiner Dashboard & Exam Management

Say to the panel: The dashboard joins exam management with live human review. Live Monitoring and History serve different stages of the exam lifecycle.

Create/manage exams, release papers and configure rules and permitted applications.

Purpose: Give the examiner control over the exam environment and start flow.

How it works: ExamManager submits the creation form with title, duration, type, paper and rules. The server persists the exam and code. Paper release changes the candidate lobby/workspace state; End Exam updates the exam and active session records.

Where to show it: dashboard/src/components/ExamManager.jsx:587 - Create Exam Modal
server/src/routes/exams.js:387 - router.patch('/:examId/status'
server/src/routes/exams.js:527 - release-paper

Live Monitoring, Active Candidates, Priority Queue and exam-scoped evidence review.

Purpose: Separate urgent review from management/history.

How it works: Dashboard maintains the exam/candidate selection. Active-exam filtering excludes completed exams. StudentList presents candidates; AlertFeed follows the selected session; PriorityQueue supports cross-candidate triage; EvidenceViewer retrieves a session history.

Where to show it: dashboard/src/pages/Dashboard.jsx:3 - selectLiveAlerts
dashboard/src/components/AlertFeed.jsx:4 - selectLiveAlerts
dashboard/src/components/EvidenceViewer.jsx:106 - EvidenceViewer

Socket.IO updates restricted by examiner ownership/admin access.

Purpose: Deliver live updates without exposing another teacher's exam.

How it works: The socket layer resolves an event's exam and emits only to the owning teacher and authorized admins. The dashboard subscribes to violations and score/review updates, while REST fetches supply initial state and refreshes.

Where to show it: server/src/sockets/violationSocket.js:43 - broadcastToExam
dashboard/src/hooks/useSocket.js:62 - socket.on('violation')

Confirm/dismiss alerts, review submissions and inspect historical exams.

Purpose: Keep an auditable human decision process.

How it works: AlertFeed calls the protected review endpoint with decision/note. Session submissions and evidence remain available through review screens. History opens completed-exam summaries, candidate rosters and exports.

Where to show it: dashboard/src/components/AlertFeed.jsx:111 - handleQuickReview
dashboard/src/components/ExamSummary.jsx:51 - ExamSummary
server/src/routes/submissions.js:9 - requireOwnedSession

Demonstration / example

Select an active exam and student, review an alert, end the exam using the confirmation dialog, then open History and inspect its summary.

Claim boundary - explain accurately

Teacher-facing access depends on authentication and ownership checks. The candidate code-lookup endpoint is a different entry flow; it is not the route used by the fixed Live End Exam action.

7. Answer Submission System

Say to the panel: The candidate views the released paper, prepares text or a file answer, then receives completion only after the server accepts the submission.

Candidate paper viewing, typed answers and supported file uploads.

Purpose: Support both written responses and practical work files.

How it works: The renderer fetches the paper through the session-aware paper endpoint. PDF uses a viewer; DOCX uses Mammoth. Text and the selected file are placed in FormData and posted to /submissions. The server stores the answer and optional file.

Where to show it: candidate-app/renderer/examScreen.js:122 - async function loadExamPaper
candidate-app/renderer/examScreen.js:1029 - async function performSubmission
server/src/routes/submissions.js:28 - router.post('/

Local text drafts, exam timing and submission controls.

Purpose: Reduce lost typing and enforce the exam workflow.

How it works: Text input writes a session-scoped localStorage draft. The countdown uses server endTime plus server-time offset and accepts extensions. Manual submission shows a confirmation; expiry attempts automatic submission. Duplicate clicks are guarded and failures allow retry.

Where to show it: candidate-app/renderer/examScreen.js:780 - function saveDraft
candidate-app/renderer/examScreen.js:14 - serverTimeOffset
candidate-app/renderer/examScreen.js:22 - submissionInFlight

Answers linked to the candidate session; examiner retrieval and download.

Purpose: Keep the correct answer with the correct candidate/exam.

How it works: The server validates sessionId, active state, text length, timing and duplicate submission. File size is limited to 15 MB. Storage URLs and submission metadata are saved against the session, and protected examiner endpoints retrieve/download them.

Where to show it: server/src/routes/submissions.js:15 - fileSize: 15
server/src/routes/submissions.js:57 - Submission.exists
server/src/routes/submissions.js:9 - requireOwnedSession

Demonstration / example

Type an answer and refresh to restore the draft. Attach a newly created file, submit, and show its record/download in examiner review.

Claim boundary - explain accurately

The old-file upload timestamp check is candidate-side. A locally saved draft is not a server submission; attached files are not automatically restored across app restarts. Answer submission is separate from offline alert buffering.

8. Audit Report / Exam Summary

Say to the panel: After an exam, the examiner can inspect the candidate roster, score distribution and summary, export structured data or print the report.

Historical exam summaries, final risk scores and violation/risk distributions.

Purpose: Summarize a completed exam for review and reporting.

How it works: The summary endpoint aggregates sessions and violations for the owned exam. ExamSummary displays totals, candidate risks and distribution charts. Scores are computed/stored by the scoring workflow; a final score is still a review metric.

Where to show it: server/src/routes/exams.js:275 - router.get('/:examId/summary'
dashboard/src/components/ExamSummary.jsx:144 - Risk distribution calculation

CSV export and browser print/Save PDF.

Purpose: Provide portable structured and printable results.

How it works: handleExportCSV builds a CSV download from the roster/summary data. The print action calls window.print(); browser print styles prepare the report and the user can select Save as PDF. It is not a dedicated server-generated PDF pipeline.

Where to show it: dashboard/src/components/ExamSummary.jsx:77 - const handleExportCSV
dashboard/src/components/ExamSummary.jsx:111 - window.print()

Detailed alert logs and screenshots remain in the separate evidence viewer; an all-evidence export is remaining work.

Purpose: Distinguish implemented reports from a complete evidence package.

How it works: Individual session evidence is accessible through EvidenceViewer and authenticated stored assets. The summary export does not assemble every screenshot/audio clip into one bundled archive/report. This is a scope boundary, not a missing summary feature.

Where to show it: dashboard/src/components/EvidenceViewer.jsx:106 - EvidenceViewer
server/src/index.js:36 - app.use('/uploads')

Demonstration / example

Open a completed exam, point to its totals and candidate roster, export CSV, then open the print preview. Open candidate evidence separately.

Claim boundary - explain accurately

Do not promise an automatically generated PDF containing all event screenshots or an all-evidence ZIP. The existing CSV and browser print features are implemented.

9. Authentication & Administration

Say to the panel: Teacher accounts and exam ownership checks protect the dashboard. Administrators have additional account, asset and audit-management screens.

Teacher/admin authentication, account verification/reset and role/ownership checks.

Purpose: Identify users and restrict privileged data/actions.

How it works: Passwords use bcrypt; login issues an access token and a refresh cookie. Verification currently uses the link strategy. Protected routes apply authentication and exam/session ownership checks. Admin routes require admin access. Reset/verification flows use the configured mail service.

Where to show it: server/src/routes/auth.js:110 - bcrypt.hash
server/src/services/verification/index.js:11 - new LinkVerificationStrategy
server/src/middleware/examAccess.js:23 - requireOwnedExam

Administrative user/asset management and activity audit records.

Purpose: Support controlled maintenance and accountability.

How it works: Admin views call endpoints for users, role changes, stored assets and audit logs. Asset deletion removes/unlinks supported files; auditLogger records selected examiner/admin actions. AuditLogView displays/filter/exports activity records.

Where to show it: server/src/routes/admin.js:656 - router.get('/users'
server/src/routes/admin.js:129 - router.get('/assets'
server/src/utils/auditLogger.js:13 - logTeacherAction

Teacher information and consistent light/dark dashboard themes.

Purpose: Make the interface identifiable and usable across lighting conditions.

How it works: AuthContext supplies teacher identity to the header/account area. App-level theme state and CSS variables drive light/dark surfaces and shared components; the controls propagate the selected theme to dashboard screens.

Where to show it: dashboard/src/context/AuthContext.jsx:37 - teacher
dashboard/src/App.jsx:28 - theme

Demonstration / example

Log in as a teacher, show the account identity and theme switch, then explain admin-only user/assets/audit screens using an authorized admin account.

Claim boundary - explain accurately

The current mail service is Ethereal, a development/test mail provider. Do not claim production email delivery. An audit log records implemented actions, not every possible OS activity.

10. Examiner-Candidate Communication

Say to the panel: The examiner can communicate instructions and manage a candidate session while the candidate remains inside the exam workflow.

Exam chat, broadcasts and examiner warnings.

Purpose: Let candidates ask questions and receive examiner instructions.

How it works: LiveExamChat and the candidate chat UI use the messages routes for session conversations and broadcasts. Examiner warnings are stored on the session; status polling detects new warnings and displays a candidate toast.

Where to show it: dashboard/src/components/LiveExamChat.jsx:11 - LiveExamChat
server/src/routes/messages.js:14 - router.
candidate-app/renderer/examScreen.js:465 - showExaminerWarningToast

Session termination and time-extension controls.

Purpose: Allow the examiner to stop a session or grant extra time.

How it works: The protected termination endpoint marks a candidate terminated and saves a reason. The renderer observes that state and displays termination handling. Exam extensions update endTime; polling adjusts the candidate countdown and displays an extension notification.

Where to show it: server/src/routes/sessions.js:394 - router.post('/:sessionId/terminate'
server/src/routes/exams.js:430 - extend-time
candidate-app/renderer/examScreen.js:427 - handleSessionTerminated

Candidate status updates and re-verification requests during the exam.

Purpose: Keep the candidate synchronized with examiner decisions.

How it works: The candidate polls session status about every three seconds for paper release, warnings, timing and camera verification. A rejected/flagged/re_verify state opens the camera re-verification modal and permits a replacement image workflow.

Where to show it: candidate-app/renderer/examScreen.js:396 - function startSessionStatusPolling
candidate-app/renderer/examScreen.js:489 - async function showCameraReverificationModal
server/src/routes/sessions.js:220 - router.patch('/:sessionId/camera-verification'

Demonstration / example

Send a warning and show it on the candidate screen, extend time and show the timer change, then demonstrate re-verification or terminate a disposable demo session.

Claim boundary - explain accurately

Candidate updates are partly polling-based and can be delayed by the poll interval or network failure. End Exam and individual candidate termination are distinct actions. Connected confirms server reachability, not every detector's health.